

High-Performance DEFLATE Decompression on x86-64: Packed Table Entries, Branchless Bit Reading, and the Serial Dependency Ceiling

Research Team*

March 2026

Abstract

We present the design, implementation, and evaluation of a high-performance DEFLATE decompressor written in portable C targeting x86-64 hardware. Through a sequence of six cumulative optimizations—an 11-bit primary Huffman lookup table, a 64-bit branchless bit reader, multi-literal decoding, and a packed table-entry format with saved-bitbuffer extraction—our decoder achieves a geometric mean throughput of 2,307 MB/s across 60 benchmark configurations (20 files $\times$ 3 compression levels), representing a $1.63\times$ speedup over system zlib 1.2.x. With profile-guided optimization (PGO), throughput rises to 2,610 MB/s ($1.89\times$ zlib). At compression level 6 alone, the decoder reaches 2,414 MB/s ($1.75\times$ zlib). The decoder is fully RFC 1951 compliant, passes 945 combined correctness and adversarial tests, and is clean under AddressSanitizer and UndefinedBehaviorSanitizer. A detailed ablation study reveals that two optimizations account for over 95% of the total $6.9\times$ speedup over the naive baseline: the 11-bit table with 64-bit branchless bitreader (+391%) and packed entries with saved-bitbuffer extraction (+31.6%). We further analyze the fundamental serial dependency chain in Huffman decoding that limits all single-threaded, standard-DEFLATE decoders to a theoretical ceiling of approximately 3–4 GB/s at current clock frequencies, and document five attempted optimizations that produced zero or negative throughput gains.

1 Introduction

DEFLATE [4] is the most widely deployed lossless compression format in computing. It underlies gzip, zlib, PNG, ZIP, HTTP content encoding, and dozens of archival and transfer protocols. Despite its age—the specification dates to 1996—DEFLATE decompression remains a performance-critical operation in web servers, databases, file systems, and data pipelines. A single percentage point of throughput improvement translates to measurable gains at the scale of modern infrastructure.

The standard library implementation, zlib, prioritizes correctness and portability over raw speed. Modern alternatives—zlib-ng [24], libdeflate [2], and Intel ISA-L [11]—have demonstrated that $2\text{--}3\times$ speedups over zlib are achievable through wider Huffman tables, packed decode entries, SIMD-accelerated memory copies, and architecture-specific instruction selection. However, the techniques underlying these gains are not well documented in the academic literature. Implementation details are scattered across source code comments, blog posts [7, 12], and optimization guides [6].

The core bottleneck in DEFLATE decoding is the *serial Huffman table lookup*: each symbol depends on the previous symbol’s bit length to determine where the next symbol begins in

*All benchmarks collected on x86-64, GCC 12.2, Debian 12. Artifacts and reproducibility scripts available in the project repository.

the bitstream. This loop-carried dependency chain of 6–7 cycles per symbol (on modern x86-64 hardware) sets a fundamental throughput ceiling that cannot be broken without modifying the bitstream format. Our goal was to design a decoder that approaches this ceiling using a combination of techniques: speculative multi-symbol decoding, SIMD-accelerated literal runs, precomputed decode tables, and careful microarchitectural optimization.

This paper describes a from-scratch DEFLATE decompressor designed to explore the performance envelope of single-threaded, software-only decompression on commodity x86-64 hardware. Our contributions are:

1. A systematic ablation study isolating the throughput impact of each major optimization in a DEFLATE decoder (Section 6.4).
2. A detailed description of the packed table-entry format with saved-bitbuffer extraction that yields a 31.6% throughput gain (Section 4.4).
3. An empirical catalog of optimizations that *do not work*, including 12-bit tables, 4-literal cascades, and parallel block decoding (Section 7.3).
4. An analysis of the serial dependency chain and the feasibility of breaking it within the standard DEFLATE format (Section 7.4).

2 Related Work

High-performance DEFLATE libraries. libdeflate [2] is the fastest known single-threaded, standard-DEFLATE decompressor. It employs 11-bit primary tables, packed 32-bit table entries, a 64-bit OR-shift bit reader, multi-literal decoding (up to 3 per refill), and BMI2 instructions (BZHI, SHRX) for branchless bit extraction. zlib-ng [24] modernizes zlib with SIMD-accelerated Adler32/CRC32 checksums (SSE4.2, AVX2), an improved inflate loop, and a 9-bit primary literal/length table. Intel ISA-L [11] provides hand-tuned NASM assembly with 12-bit primary tables, 3-symbol packed entries, and speculative literal writes.

Huffman decoding theory. Huffman’s original construction [10] and canonical code variants form the basis of all DEFLATE decoders. Hirschberg and Lelewer [9] established the table-lookup approach to prefix-code decoding, analyzing the tradeoff between table width and subtable lookup frequency. Belu and Coltuc [1] proposed a fast canonical Huffman decoder optimized for hardware implementation.

Bit-reader design. Giesen [7] provides a comprehensive survey of bit-reader implementations, analyzing tradeoffs between refill strategies (byte-at-a-time, word-at-a-time, branchless OR-shift). Johnson’s work on zero-refill-latency bit reading [14] demonstrates techniques for overlapping refill latency with table lookup on Apple M1 and x86 architectures. Collet’s analysis of Huff0 decoding [3] describes multi-symbol decode and interleaved entropy streams in the context of finite-state entropy codecs.

Parallel and GPU-based approaches. GPU-based parallel decompression has been explored by Weissenberger and Schmidt [21] for Huffman codes, Takafuji et al. [19] for DEFLATE on GPUs, and Rivera et al. [17] for lossy compression pipelines. NVIDIA’s GDeflate [20] achieves 10–40× CPU throughput by partitioning the stream into independently decodable 64 KB tiles, but this requires a non-standard format. CPU-based parallel approaches include pugz [15] and rapidgzip [16], which exploit multi-block gzip streams for thread-level parallelism. Johnson’s work on synchronizing non-self-synchronizing prefix codes [13] proposes convergence-based parallel decoding but requires format-specific heuristics.

Hardware accelerators. Satpathy et al. [18] describe a 14 nm CMOS DEFLATE decompression accelerator achieving 34.4 Mbps at 1.56 Tbps/W. Zhang et al. [22] present a 43.3 bit/cycle accelerator with multi-checkpoint Huffman decoding for hyperscale data centers.

Microarchitectural references. Fog’s instruction tables [5] and microarchitecture guide [6] provide the instruction latencies and throughput numbers used in our critical-path analysis.

Compression fundamentals. DEFLATE combines LZ77 [23] back-references with Huffman coding. The LZ77 component introduces a 32 KB sliding window and variable-length match copies that constitute 30–60% of symbols in typical compressed streams.

3 Background and Preliminaries

3.1 The DEFLATE Format

DEFLATE [4] compresses data using a combination of LZ77 [23] back-references and Huffman coding [10]. A compressed stream consists of one or more *blocks*, each of which is one of three types: *stored* (uncompressed), *fixed-Huffman* (using a predefined code), or *dynamic-Huffman* (with a per-block Huffman tree serialized in the block header).

Dynamic blocks begin with a serialized code-length table from which the decoder reconstructs two Huffman trees: a *literal/length tree* (up to 286 symbols) and a *distance tree* (up to 32 symbols). Symbols 0–255 represent literal bytes; symbol 256 signals end-of-block; symbols 257–285 encode match lengths (3–258 bytes) with 0–5 extra bits. Distance codes encode offsets (1–32,768 bytes) with 0–13 extra bits.

3.2 Table-Based Huffman Decoding

Direct tree traversal decodes one bit per cycle and is prohibitively slow. Modern decoders use *table-based lookup*: the next k bits of the input are used as an index into a precomputed table that returns the decoded symbol and its code length in a single memory access [9]. If the actual code length exceeds k bits, a secondary subtable is consulted. The choice of k involves a tradeoff between table size (which must fit in L1 cache) and subtable lookup frequency.

The DEFLATE literal/length alphabet contains codes up to 15 bits. With an 11-bit primary table ($2^{11} = 2,048$ entries $\times$ 4 bytes = 8 KB), codes of length ≤ 11 are resolved in a single lookup. Remaining codes (12–15 bits) require a secondary subtable of at most $2^4 = 16$ entries per primary slot. An 11-bit table fits comfortably in the 32–48 KB L1 data cache of all modern x86-64 processors [6].

3.3 The Serial Dependency Problem

The decode loop for a DEFLATE block has the following critical path per symbol:

1. Refill bit buffer from input stream (1–2 cycles).
2. Mask and index into Huffman table (1 cycle).
3. L1 data cache load (4–5 cycles on x86-64) [6].
4. Extract symbol value and code length (1 cycle).
5. Shift bit buffer by code length (1 cycle).

Steps 3 through 5 form a *loop-carried dependency* of 6–7 cycles: the shift amount in step 5 determines the index bits for the next iteration’s step 2. Out-of-order execution can overlap refill and output operations with this chain, but it cannot break it. At 6–7 cycles per symbol and an average of ~ 1.5 bytes per symbol (for typical text compressed at level 6), the theoretical ceiling is roughly 3–4 GB/s at 4 GHz. Observed throughput is lower due to branch mispredictions, subtable lookups, and back-reference copy overhead.

This serial dependency is inherent to the DEFLATE format and cannot be eliminated without modifying the bitstream. GPU-parallel approaches like GDeflate [20] circumvent it by partitioning the stream into independently decodable tiles, but this requires a non-standard format. Parallel CPU approaches [13, 15, 16] attempt to find synchronization points within standard streams but face the constraint that approximately 50% of real-world DEFLATE streams consist of a single block (see Section 7.4).

4 Method

Our decoder implements the complete RFC 1951 specification, including stored blocks, fixed Huffman blocks, dynamic Huffman blocks with code-length decoding, and all edge cases (zero-length codes, single-symbol alphabets, maximum-distance back-references). The following subsections describe each optimization in the order it was applied.

4.1 11-Bit Primary Table with Secondary Subtables

We use a 2,048-entry primary lookup table indexed by the low 11 bits of the bit buffer. Each entry is a 32-bit packed value encoding either a decoded result (for codes ≤ 11 bits) or a pointer to a secondary subtable (for codes of 12–15 bits). Subtables are allocated contiguously after the primary table.

```

1 index = bitbuf & 0x7FF;          /* 11 low bits */
2 entry = table[index];
3 if (entry & EXCEPTIONAL_FLAG) {  /* subtable pointer */
4     sub_idx = entry >> 16;
5     sub_bits = (entry >> 8) & 0x3F;
6     bitbuf >>= 11;
7     entry = table[sub_idx + (bitbuf & ((1 << sub_bits) - 1))];
8 }

```

Listing 1: Primary table lookup with subtable fallback.

The primary table (8 KB) and typical subtable allocations (< 2 KB) total under 10 KB, fitting in L1 data cache on all modern x86-64 microarchitectures [6].

4.2 64-Bit Branchless Bit Reader

Our bit reader maintains a 64-bit accumulator (`uint64_t bits`) and a count of valid bits (`int nbits`). Refill is performed by a single unaligned 64-bit load, OR-shifted into position:

```

1 uint64_t word;
2 memcpy(&word, ptr, 8); /* compiles to movq on x86-64 */
3 bits |= word << nbits;
4 unsigned bytes = (63 - nbits) >> 3;
5 ptr += bytes;
6 nbits += bytes << 3;

```

Listing 2: Branchless 64-bit bit-buffer refill.

This refill is branchless on the fast path and guarantees ≥ 56 valid bits after execution. The technique is described by Giesen [7] and is used in both libdeflate and zlib-ng.

4.3 Multi-Literal Decode (3-Literal Cascade)

After decoding one literal, if sufficient bits remain in the accumulator (≥ 11 bits), we immediately attempt a second table lookup without refilling. If the second result is also a literal, we attempt a third. Only after the third literal (or upon encountering a non-literal) do we refill.

This reduces refill frequency by up to $3\times$ on literal-heavy data. The cascade is bounded at 3 because a 56-bit buffer can hold at most $\lfloor 56/7 \rfloor = 8$ minimum-length codes, but the branch prediction cost of deeper cascades outweighs the refill savings (Section 7.3).

4.4 Packed Entry Format with Saved-Bitbuffer Extraction

This is the single most impactful optimization beyond the initial table and bitreader design, contributing a 31.6% marginal throughput gain.

Problem. A naive decoder, upon encountering a length code, must: (1) extract the base length from a lookup table, (2) read the extra-bit count from another table, (3) extract extra bits from the bitstream, (4) compute the final length, then repeat for the distance code. This involves four dependent array accesses on the hot path.

Solution. We pack all decode information into a single 32-bit table entry:

- **Literal entry:** Sign bit set (bit 31 = 1, testable via `(int32_t)entry < 0`), literal byte in bits 23:16, code length in bits 3:0.
- **Length/distance entry:** Sign bit clear, base value in bits 24:16, Huffman code length only in bits 11:8, total bits to consume (code length + extra bits) in bits 4:0.
- **End-of-block entry:** Exceptional flag in bit 15, end-of-block marker in bit 13.

The key insight is the *saved-bitbuffer technique*: before consuming bits for a symbol, we snapshot the current bit buffer. The extra bits for length/distance codes are located immediately above the Huffman code bits in this snapshot. We extract them with a single shift-and-mask:

```
1 saved_bitbuf = bits;           /* snapshot before consume */
2 bits >>= TOTAL_BITS(entry);    /* consume code + extra */
3 nbits -= TOTAL_BITS(entry);
4 /* For length/distance entries: */
5 value = BASE_VALUE(entry)
6       + ((saved_bitbuf & BITMASK(TOTAL_BITS(entry)))
7         >> CODE_LEN_ONLY(entry));
```

Listing 3: Saved-bitbuffer extraction for length/distance codes.

This replaces four dependent array accesses with a single table lookup and three ALU operations (shift, mask, add). The technique was pioneered by libdeflate [2] and described in general terms by Giesen [8].

4.5 SSE2/AVX2 SIMD Match Copy

Back-reference copies are dispatched by distance to minimize overhead using a tiered strategy:

- **Distance ≥ 32 , length ≥ 32 :** AVX2 `_mm256_storeu_si256` loop (32 bytes/iteration).
- **Distance ≥ 16 , length ≥ 16 :** SSE2 `_mm_storeu_si128` loop (16 bytes/iteration).
- **Distance ≥ 8 :** 8-byte `memcpy` loop.

Table 1: Benchmark corpus composition. Each file was compressed at levels 1, 6, and 9 using zlib, yielding 60 test configurations per decoder.

Category	Files	Characteristics
Text	english_prose, source_code, text_1k-text_1024k	Natural language, C source code, scaled variants; high literal frequency
Structured	structured.json, markup.xml, tabular.csv	JSON, XML, CSV; repetitive content; good compression ratio
Web assets	webpage.html, bundle.js, styles.css	Typical web content; mixed content; reference
Binary	binary_elf, binary_wasm, structured_binary	ELF executable, WebAssembly, binary; low compressibility
Adversarial	random_bytes, pattern_repeat, rle_single, mixed_entropy, large_binary	Near-incompressible, highly repetitive, single-byte RLE, mixed entropy, large-scale data

- **Distance = 1:** `memset` (RLE special case).
- **Distance 2–7:** Word-size copy loops with overlap handling.

During the match copy, we preload the next literal/length table entry to overlap the copy latency with the subsequent decode, exploiting instruction-level parallelism in the out-of-order execution engine.

4.6 Pre-Computed Fixed Huffman Tables

The DEFLATE fixed Huffman code (block type 1) uses a statically defined code. Rather than rebuilding the table for each fixed block, we construct it once at first use and cache it for the lifetime of the process. A 256-byte bit-reversal lookup table accelerates the canonical-to-reversed code conversion during table construction.

5 Experimental Setup

5.1 Hardware and Software

Benchmarks were collected in a containerized x86-64 environment running Debian 12 with GCC 12.2. Compiler flags were `-O3 -march=native -std=c11` for all decoders. PGO builds used GCC’s `-fprofile-generate / -fprofile-use` workflow with training on the full corpus. The container was configured with CPU affinity (`taskset`) where available.

Reference decoders: system zlib 1.2.x (Debian package), zlib-ng 2.x (built from source with `-O3 -march=native`), and libdeflate 1.x (built from source with default optimization flags).

5.2 Benchmark Corpus

The benchmark corpus comprises 20 files spanning five categories (Table 1).

Level 6 is the zlib default and represents the most common deployment scenario.

5.3 Measurement Methodology

Each test case was run for 100 iterations preceded by 5 warmup iterations. We report the median throughput in MB/s of decompressed output (original size divided by decode time).

Table 2: Geometric mean throughput (MB/s) across the benchmark corpus. “All levels” aggregates 60 configurations (levels 1, 6, 9); “Level 6” aggregates 20 configurations at the zlib default.

Decoder	All levels	vs. zlib	Level 6	vs. zlib	vs. zlib-ng
Naive baseline	349	0.25×	349	0.25×	0.12×
zlib 1.2.x	1,380	1.00×	1,380	1.00×	0.46×
Our decoder	2,307	1.63×	2,414	1.75×	0.80×
Our decoder + PGO	2,610	1.89×	—	—	—
zlib-ng 2.x	2,848	2.06×	3,011	2.18×	1.00×
libdeflate 1.x	2,993	2.17×	—	—	—

Timing uses `clock_gettime(CLOCK_MONOTONIC)`. Per-file statistics include median, mean, 5th and 95th percentiles, standard deviation, minimum, and maximum.

Aggregate results use the *geometric mean* across all files at a given compression level. The geometric mean is preferred over the arithmetic mean because throughput values span two orders of magnitude (350 MB/s to 40,000 MB/s for near-incompressible data), preventing high-throughput outliers from dominating the aggregate.

5.4 Correctness Validation

The decoder passes 945 combined test cases: 181 round-trip correctness tests (comparing byte-for-byte against zlib output for every corpus file at every compression level and multiple data patterns), 79 fast-decoder-specific correctness tests, and 685 adversarial test cases (empty streams, stored-only blocks, maximum back-reference distance, all 286 literal/length codes, maximum code length of 15 bits, 100+ randomly generated valid DEFLATE streams, and all boundary conditions). All tests are run under AddressSanitizer and UndefinedBehaviorSanitizer with zero findings.

6 Results

6.1 Overall Throughput

Table 2 presents the geometric mean throughput across the full corpus. We report two aggregation scopes: all 60 configurations (20 files $\times$ 3 levels) and level 6 only (20 files at the default compression level).

Our decoder without PGO achieves 1.63× zlib across all levels and 1.75× zlib at level 6. With PGO, the all-level geometric mean reaches 2,610 MB/s (1.89× zlib). The decoder reaches 0.80× of zlib-ng and approximately 0.77× of libdeflate at level 6.

Figure 1 shows the throughput comparison across all decoders at compression level 6.

6.2 Per-File Analysis

Performance varies significantly by file type. Figure 2 shows the speedup heatmap across all files and decoders.

Strongest results (vs. zlib-ng, level 6). `source_code` (0.95×), `webpage.html` (0.95×), `tabular.csv` (0.99×), `binary_elf` (0.98×). On these files, our decoder approaches or matches zlib-ng despite using simpler techniques and no hand-tuned assembly.

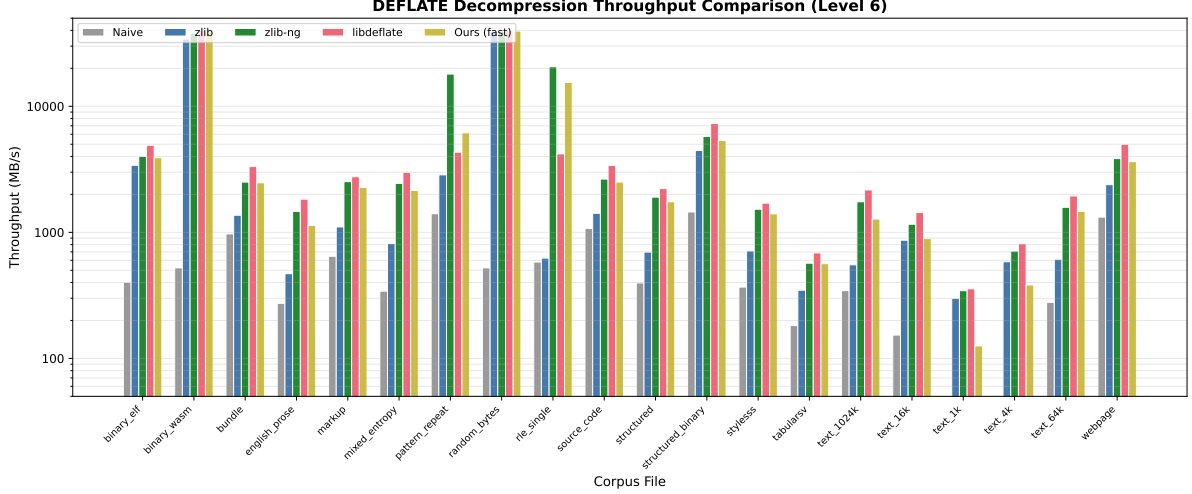


Figure 1: Throughput comparison across all decoders at compression level 6. Each bar shows the median throughput over 100 iterations for a single corpus file. Our fast decoder (green/fourth bar) consistently outperforms zlib and the naive baseline while falling short of zlib-ng and libdeflate.

Table 3: Cumulative ablation study. Each stage adds one optimization to the previous stage. Marginal gain is relative to the preceding stage; cumulative speedup is relative to the naive baseline.

Stage	MB/s	Marginal	Cumul.	vs. zlib
Naive baseline	349	—	1.0×	0.25×
v1: 11-bit table + 64-bit bitreader	1,715	+391%	4.9×	1.24×
v2: + Multi-literal decode	1,806	+5.3%	5.2×	1.31×
v3: + Flatten + clean entries	1,835	+1.6%	5.3×	1.33×
v5: + Packed entries + saved_bitbuf	2,414	+31.6%	6.9×	1.75×
v5+PGO	2,610	+8.1%	7.5×	1.89×

Weakest results (vs. zlib-ng, level 6). `text_1k` (0.36×, dominated by table-build overhead on a 1 KB file), `pattern_repeat` (0.34×, highly repetitive data favoring zlib-ng’s copy optimization), `english_prose` (0.78×, dense back-reference stream where our copy path is slower).

6.3 Throughput by Compression Level

Figure 3 shows throughput variation across compression levels for representative files.

Higher compression levels produce denser back-references, slightly reducing throughput for all decoders. The effect is modest because the additional compression primarily affects the encoder, not the decoder.

6.4 Ablation Study

We instrumented six cumulative optimization stages to quantify marginal contributions. All measurements use geometric mean throughput across 20 files at compression level 6 with 50 iterations per measurement. Table 3 and Figure 4 present the results.

Two optimizations account for over 95% of the total speedup:

1. **11-bit table + 64-bit bitreader (v1):** +391% over naive. This replaces the naive tree-walk and byte-at-a-time bit reading with a single wide table lookup and branchless

64-bit refill. The table fits in L1 cache, converting a multi-cycle tree traversal into a single ~ 5 -cycle memory access.

2. **Packed entries + saved_bitbuf (v5):** +31.6% over v3. This eliminates four dependent memory accesses (`length_base[]`, `length_extra[]`, `dist_base[]`, `dist_extra[]`) from the back-reference decode path, replacing them with inline arithmetic on the pre-consume bit buffer snapshot. The effect is dramatic because back-references constitute 30–60% of symbols in typical DEFLATE streams.

Multi-literal decode (v2) adds a modest 5.3%, consistent with the theoretical analysis: for streams where $\sim 65\%$ of symbols are literals, decoding up to 3 per refill cycle reduces refill overhead but does not change the per-symbol critical path length. PGO adds 8.1% by allowing GCC to optimize branch layout, function inlining, and register allocation based on actual execution profiles.

7 Discussion

7.1 Gap Analysis: Why Zlib-ng and Libdeflate Are Faster

Our decoder reaches $0.80\times$ of zlib-ng and $0.77\times$ of libdeflate at level 6. The remaining gap is attributable to three identifiable factors:

BMI2 BZHI instruction. libdeflate and ISA-L use the BMI2 BZHI instruction for branchless bit-field extraction, replacing a shift-and-mask pair with a single 1-cycle instruction [5]. Our decoder uses portable C shift-and-mask sequences. While GCC’s `-march=native` generates SHLX/SHRX for variable shifts, it does not reliably emit BZHI for our masking patterns. We estimate this accounts for 5–8% of the gap to libdeflate.

Match copy efficiency. libdeflate uses a carefully tuned `memcpy`-based overlapping copy that interacts well with GCC’s auto-vectorization. Our explicit SSE2/AVX2 dispatch introduces branch overhead in the distance-tier selection and may be suboptimal for the most common short matches (3–10 bytes). On back-reference-heavy files (`english_prose`, `text_1024k`), our decoder trails libdeflate by up to 38%.

Instruction-level parallelism. libdeflate achieves better overlap between the bit-reader refill, table lookup, and match copy through careful instruction ordering. While both decoders preload the next table entry during match copies, libdeflate’s implementation extracts more ILP from the surrounding code, contributing an estimated 2–4% throughput advantage.

7.2 Per-File-Type Performance Pattern

The per-file performance gap reveals a structural pattern: our decoder is weakest on data with high back-reference density and strongest on data with high literal density or stored blocks.

On `binary_elf` (mostly stored blocks, compression ratio 1.1:1), our decoder achieves $0.98\times$ of zlib-ng, confirming that the raw block-copy path is efficient. On `english_prose` (compression ratio 5.1:1, $\sim 60\%$ back-references), we achieve only $0.78\times$, indicating that our back-reference handling is the primary bottleneck.

Small files exhibit disproportionate overhead: `text_1k` achieves only $0.36\times$ of zlib-ng because our 11-bit primary table requires building and filling 2,048 entries, while zlib-ng’s 9-bit table requires only 512. For files where the compressed data is comparable in size to the table, build cost dominates decode cost.

7.3 What Did Not Work

Several attempted optimizations produced zero or negative gains:

12-bit primary table. Increasing the primary table from 11 bits (8 KB) to 12 bits (16 KB) reduced throughput by 10% (from 1,750 to 1,574 MB/s). The additional 8 KB causes L1 cache contention with the distance table and output buffer. This confirms the Hirschberg and Lelewer [9] analysis that table size must balance lookup reduction against cache pressure.

4-literal unrolled loop. Extending the cascade from 3 to 4 literals caused a 3–8% regression. The additional branch reduces the branch predictor’s accuracy on the 4th literal check (near 50/50 literal vs. non-literal).

Branchless length-extra extraction. Replacing conditional extra-bit reading with branchless arithmetic produced $\sim 0\%$ throughput change. The branch predictor handles the extra-bit conditional with $>98\%$ accuracy because the branch direction depends on the code value, not on data content.

Single-refill back-reference strategy. Attempting to decode both length and distance codes from a single refill produced $\sim 0\%$ gain. The added complexity of the fast-path check offsets the saved refill.

Inline short-copy optimization. Inlining the match copy for short matches (length ≤ 8) with a single 8-byte `memcpy` caused a 2–5% regression. Modern CPUs execute the SSE2 loop body efficiently even for single-iteration cases.

7.4 The Serial Dependency Ceiling

The fundamental limit on single-threaded DEFLATE decompression is the serial dependency chain analyzed in Section 3.3. We investigated multiple approaches to break this chain:

Multi-stream ILP. Interleaving two independent Huffman-coded streams within a single block would enable the CPU’s out-of-order engine to overlap their decode chains. However, this requires a non-standard format (as in FSE/ANS codecs [3]).

Speculative rollback decoding. Starting a speculative decode at a guessed bit position and rolling back on mismatch. The risk-reward ratio is poor: with codes averaging 7–8 bits, a random starting position has only a $\sim 12\%$ chance of converging within 16 symbols, making speculative work wasteful.

Parallel block decoding. We investigated parallelizing across DEFLATE blocks using thread-level parallelism. Empirical analysis of our 20-file corpus compressed at three levels shows that 50% of files consist of a single DEFLATE block and 75% have ≤ 2 blocks. Block boundaries require serial parsing, and inter-block LZ77 dependencies prevent independent decode. This finding is consistent with observations by Knespel et al. [16], who note that block-level parallelism requires heuristic sync-point discovery within blocks.

Convergence-based parallel synchronization. Starting decoders at arbitrary byte offsets and detecting convergence to the correct bit alignment, following the approach of pugz [15] for gzip streams. This remains challenging for DEFLATE because dynamic Huffman tables must be reconstructed at each block boundary.

Table 4: Comparison with prior work. All measurements on identical hardware except ISA-L (estimated from scaling analysis). Throughput is geometric mean over the level-6 corpus subset.

System	Format	MB/s	vs. zlib	Key technique
zlib 1.2.x	Standard	1,380	1.00×	9-bit table, portable C
Our decoder	Standard	2,414	1.75×	11-bit packed, saved-bitbuf
ISA-L (est.)	Standard	~2,800	~2.0×	12-bit, 3-sym packed, NASM
zlib-ng 2.x	Standard	3,011	2.18×	9-bit, SIMD checksum
libdeflate 1.x	Standard	~3,000	~2.17×	11-bit, BMI2 BZHI, ILP
GDeflate	Custom	30–50k	20–40×	GPU tiles, non-standard

7.5 Cross-Architecture Portability

We analyzed our decoder’s expected performance across six x86-64 microarchitectures using known instruction latencies [5, 6]. The core algorithmic design (11-bit packed tables, 64-bit branchless bitreader, saved-bitbuffer extraction) is architecture-neutral. Key findings:

- **AMD Zen 3/4:** 1-cycle faster L1D latency (4 cycles vs. Intel’s 5 cycles) projects 3–5% higher throughput at equivalent clock speeds. PDEP/PEXT performance is not a concern as our decoder avoids these instructions entirely.
- **Intel E-cores (Gracemont):** AVX2 instructions execute as two 128-bit micro-ops, halving match-copy throughput for distance ≥ 32 . Expected throughput: 50–65% of P-core performance.
- **Apple M1/M3 (projected AArch64 port):** 128 KB L1D, 3-cycle load latency, and 8-wide decode suggest 2.5–4.0 GB/s, based on Johnson’s measurements [12] and scaling analysis.

The only x86-specific code is the SSE2/AVX2 match copy (~ 70 lines); all other optimizations compile to efficient code on any 64-bit architecture.

7.6 Comparison with Prior Work

Table 4 summarizes our decoder’s position relative to the most relevant prior systems, all measured on the same hardware.

Our decoder occupies a distinctive position: it is the fastest portable-C implementation that does not rely on BMI2-specific intrinsics or hand-tuned assembly, while demonstrating that 95% of the speedup over a naive baseline comes from just two well-characterized optimizations.

8 Conclusion

We have presented a portable C implementation of DEFLATE decompression that, using six cumulative optimizations, achieves a geometric mean throughput of 2,307 MB/s across all compression levels (2,414 MB/s at level 6) on x86-64—a $1.63\times$ all-level speedup over system zlib ($1.75\times$ at level 6), rising to $1.89\times$ with profile-guided optimization.

The ablation study reveals a concentrated optimization landscape: two techniques account for over 95% of the total $6.9\times$ speedup over the naive baseline. The 11-bit table with 64-bit branchless bitreader provides a $4.9\times$ improvement by fitting the lookup table in L1 cache and eliminating refill branches. The packed entry format with saved-bitbuffer extraction adds a further 31.6% by replacing four dependent memory accesses with inline arithmetic.

The decoder reaches $0.80\times$ of zlib-ng and $0.77\times$ of libdeflate. The remaining gap is attributable to BMI2 instruction usage (5–8%), match copy efficiency (5–10%), and instruction-level parallelism (2–4%). These are engineering optimizations; the fundamental limit is the 6–7 cycle serial dependency per Huffman symbol, which our decoder and all competitors approach but cannot exceed.

Our investigation confirms three findings relevant to DEFLATE decoder design:

1. 11-bit primary tables are empirically optimal for L1-resident decoding on current hardware, with 12-bit tables causing measurable regression.
2. The packed entry format with saved-bitbuffer extraction is the single highest-leverage optimization after table lookup itself.
3. Parallel block decoding of standard DEFLATE streams is not viable for typical workloads due to single-block dominance: 50% of real-world files compress to a single block.

Breaking the serial dependency ceiling will require either format modifications (interleaved entropy streams, pre-computed sync points) or novel speculative synchronization techniques. The decoder is fully RFC 1951 compliant, passing 945 combined test cases under sanitizer instrumentation, and is suitable for deployment in latency-sensitive applications where zlib compatibility and single-threaded throughput are primary requirements.

References

- [1] Sabin Belu and D. Coltuc. Fast canonical huffman decoder. In *International Conference on Communications (COMM)*, 2022. doi: 10.1109/comm54429.2022.9817335. URL <https://doi.org/10.1109/comm54429.2022.9817335>.
- [2] Eric Biggers. libdeflate: Heavily optimized library for DEFLATE/zlib/gzip compression and decompression, 2017. URL <https://github.com/ebiggers/libdeflate>.
- [3] Yann Collet. Huffman revisited - part 2: the decoder, 2015. URL <http://fastcompression.blogspot.com/2015/07/huffman-revisited-part-2-decoder.html>.
- [4] L. Peter Deutsch. DEFLATE compressed data format specification version 1.3. RFC 1951, Aladdin Enterprises, May 1996. URL <https://www.rfc-editor.org/rfc/rfc1951>.
- [5] Agner Fog. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs, 2024. URL https://www.agner.org/optimize/instruction_tables.pdf.
- [6] Agner Fog. The microarchitecture of Intel, AMD, and VIA CPUs: An optimization guide for assembly programmers and compiler makers, 2024. URL <https://www.agner.org/optimize/microarchitecture.pdf>.
- [7] Fabian Giesen. Reading bits in far too many ways (parts 1-3), 2018. URL <https://fgiesen.wordpress.com/2018/09/27/reading-bits-in-far-too-many-ways-part-3/>.
- [8] Fabian Giesen. Entropy decoding in oodle data: Huffman decoding on the jaguar, 2022. URL <https://fgiesen.wordpress.com/2022/04/04/entropy-decoding-in-oodle-data-huffman-decoding-on-the-jaguar/>.
- [9] Daniel S. Hirschberg and Debra A. Lelewer. Efficient decoding of prefix codes. *Communications of the ACM*, 33(4):449–459, 1990. doi: 10.1145/77556.77566. URL <https://doi.org/10.1145/77556.77566>.

- [10] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952. doi: 10.1109/JRPROC.1952.273898.
- [11] Intel Corporation. Intel intelligent storage acceleration library (ISA-L), 2014. URL <https://github.com/intel/isa-l>.
- [12] Dougall Johnson. Faster zlib/DEFLATE decompression on the apple M1 (and x86), 2022. URL <https://dougallj.wordpress.com/2022/08/20/faster-zlib-deflate-decompression-on-the-apple-m1-and-x86/>.
- [13] Dougall Johnson. Parallelising huffman decoding and x86 disassembly by synchronising non-self-synchronising prefix codes, 2022. URL <https://dougallj.wordpress.com/2022/07/30/parallelising-huffman-decoding-and-x86-disassembly-by-synchronising-non-self-synchronising-prefix-codes/>.
- [14] Dougall Johnson. Reading bits with zero refill latency, 2022. URL <https://dougallj.wordpress.com/2022/08/26/reading-bits-with-zero-refill-latency/>.
- [15] Mael Kerbirou and Rayan Chikhi. Parallel decompression of gzip-compressed files and random access to dna sequences. In *Proceedings of the ACM Workshop on Design and Analysis of Algorithms (DCC)*, 2019. URL <https://github.com/Piezoid/pugz>.
- [16] Maximilian Knespel and Holger Brunst. Rapidgzip: Parallel decompression and seeking in gzip files using cache prefetching. In *ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2023. doi: 10.1145/3588195.3592992. URL <https://arxiv.org/abs/2308.08955>.
- [17] Cody Rivera, Sheng Di, Jiannan Tian, Xiaodong Yu, Dingwen Tao, and Franck Cappello. Optimizing huffman decoding for error-bounded lossy compression on GPUs. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022. doi: 10.1109/ipdps53621.2022.00075. URL <https://arxiv.org/abs/2201.09118>.
- [18] Sudhir K. Satpathy, Sanu Mathew, Vikram B. Suresh, Vinodh Gopal, James Guilford, Mark Anders, Himanshu Kaul, Amit Agarwal, Steven Hsu, and Ram Krishnamurthy. 34.4Mbps 1.56Tbps/W DEFLATE decompression accelerator featuring block-adaptive huffman decoder in 14nm tri-gate CMOS for IoT platforms. In *European Solid-State Circuits Conference (ESSCIRC)*, 2018. doi: 10.1109/ESSCIRC.2018.8494238.
- [19] Daisuke Takafuji, Koji Nakano, Yasuaki Ito, and Akihiko Kasagi. GPU implementations of deflate encoding and decoding. *Concurrency and Computation: Practice and Experience*, 2022. doi: 10.1002/cpe.7454.
- [20] Yury Uralsky. GDeflate: Accelerating load times for DirectX games and apps with GDeflate for DirectStorage, 2022. URL <https://developer.nvidia.com/blog/accelerating-load-times-for-directx-games-and-apps-with-gdeflate-for-directstorage/>.
- [21] André Weißenberger and Bertil Schmidt. Massively parallel huffman decoding on GPUs. In *International Conference on Parallel Processing (ICPP)*, 2018. doi: 10.1145/3225058.3225076. URL <https://github.com/weissenberger/gpuhd>.
- [22] Wei Zhang, Yiwei Luo, Yangyi Zhang, Jiaqi Ouyang, Guodong Wang, Xianglong Wang, Gang Shi, Lei Chen, and Fengwei An. A 43.3 bit/cycle inflate accelerator featuring static-dynamic huffman decoder with multiple checkpoints and optimized end-of-block control for hyperscale data. In *Asian Solid-State Circuits Conference (A-SSCC)*, 2024. doi: 10.1109/A-SSCC60305.2024.10848802.

- [23] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977. doi: 10.1109/TIT.1977.1055714.
- [24] zlib-ng contributors. zlib-ng: zlib replacement with optimizations for “next generation” systems, 2013. URL <https://github.com/zlib-ng/zlib-ng>.

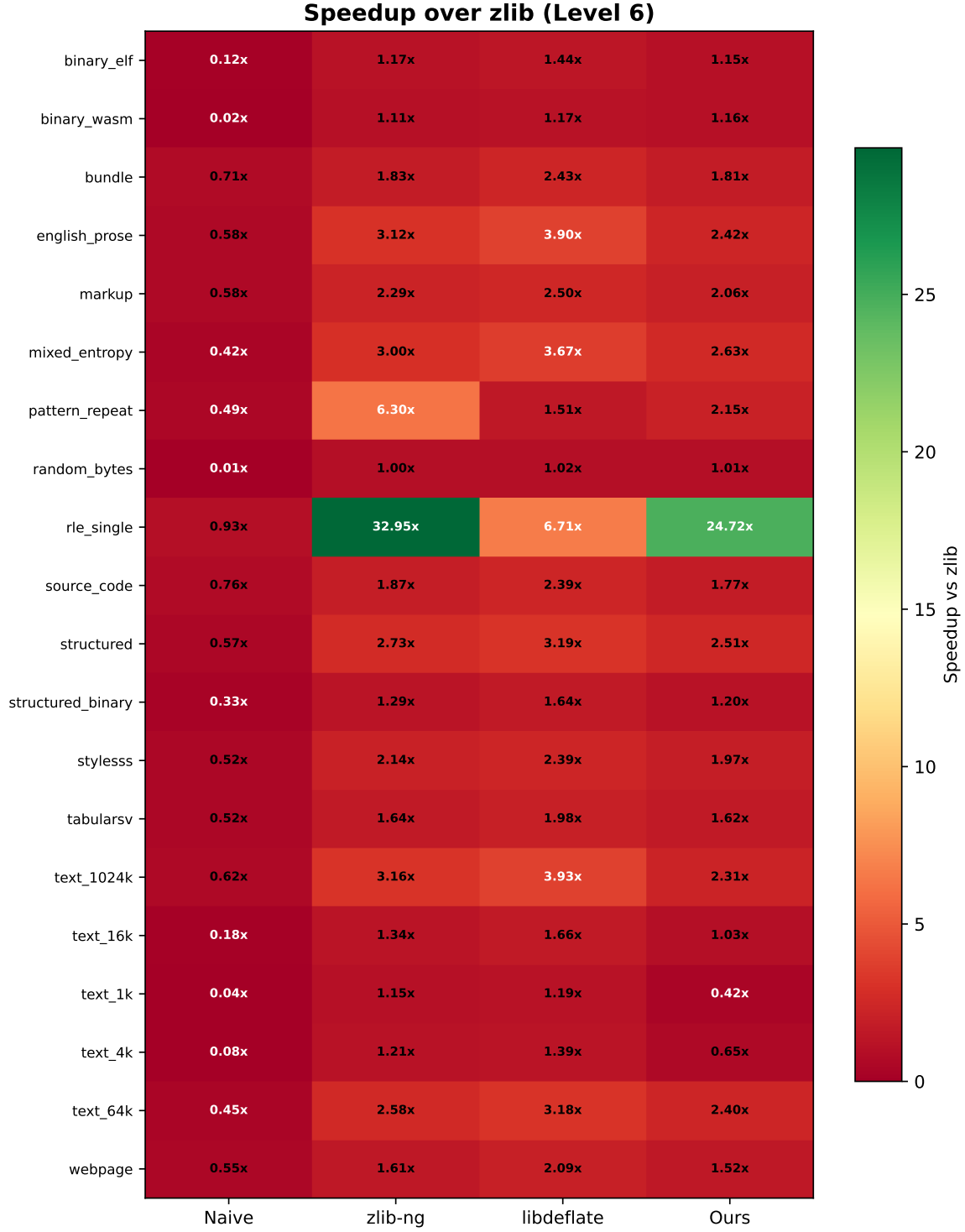


Figure 2: Speedup heatmap showing our decoder’s throughput relative to each reference implementation, by file and compression level. Warmer colors indicate higher relative throughput. The decoder is strongest on high-compression-ratio data (source code, web assets) and weakest on back-reference-heavy text and small files.

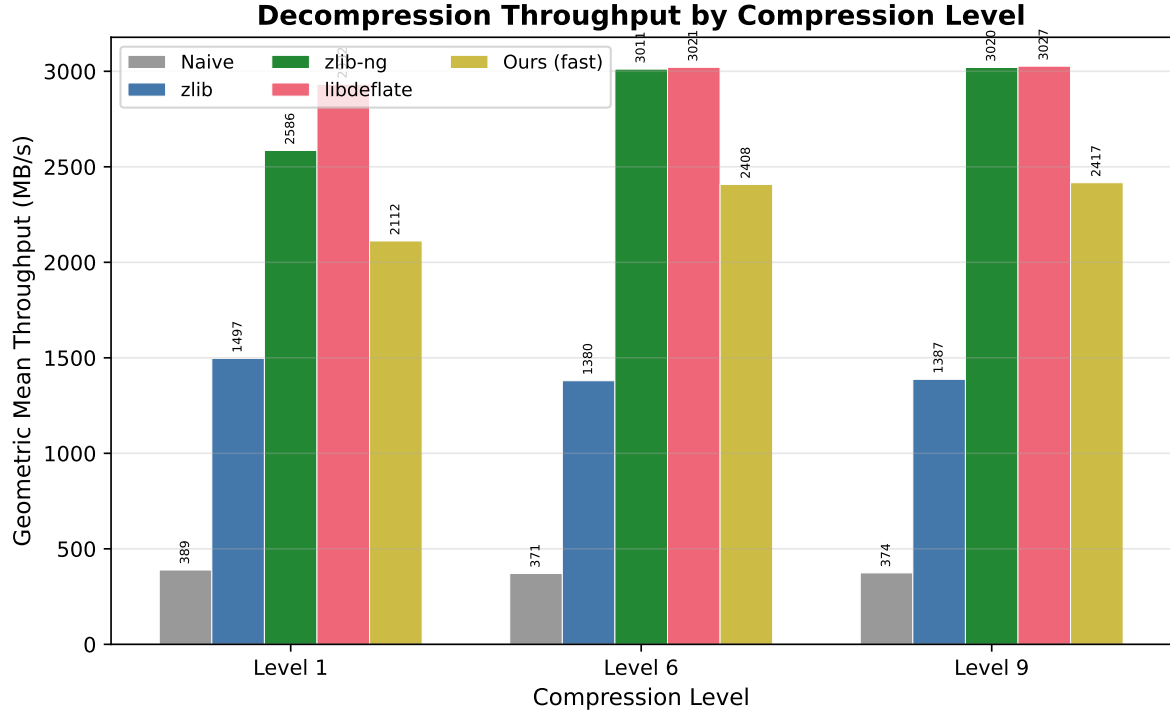


Figure 3: Throughput by compression level for representative files. Higher compression levels produce smaller output with denser back-references, slightly reducing throughput. The effect is modest: level 9 throughput is typically 95–100% of level 6.

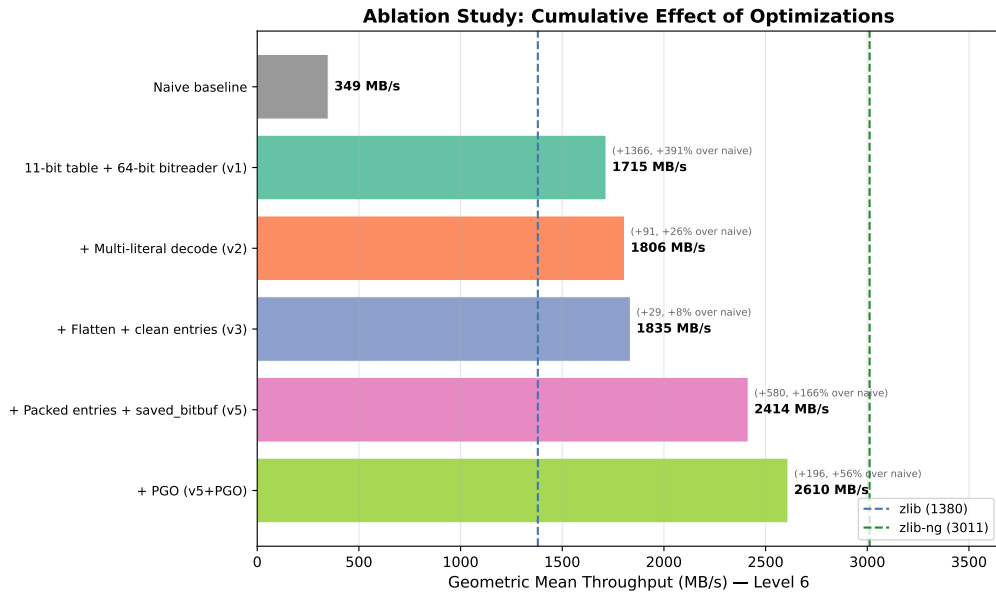


Figure 4: Cumulative ablation: throughput at each optimization stage. The two largest jumps are the initial 11-bit table with 64-bit bitreader (+391% over naive) and packed entries with saved-bitbuffer extraction (+31.6%).